N L P P O R T F O L I O P R O J E C T

Extractive Text Summarization Using PageRank

Building a Graph-Based Sentence Ranking System for Presidential Inaugural Speeches

Tools	Python, NLTK, scikit-learn, NetworkX, Pandas
Domain	NLP, Graph Analytics, Information Retrieval
Dataset	Trump 2017 Inaugural Address (90 sentences)
Techniques	TF-IDF, Gramian Matrix, PageRank, Graph Theory

Executive Summary

This project implements extractive text summarization using Google's PageRank algorithm applied to a sentence similarity graph. Starting from Trump's 2017 inaugural address (90 sentences), the system builds a TF-IDF document-term matrix, computes a Gramian matrix of inter-sentence correlations, constructs a weighted graph, and applies PageRank to identify the most "important" sentences — those most connected to and representative of the speech as a whole.

The project demonstrates proficiency in NLP text representation, linear algebra (Gramian/cosine similarity), graph theory, network analytics, and the application of classic algorithms (PageRank) to natural language problems.

Problem Statement

Given a single document (a presidential inaugural speech), the goal is to automatically identify the most important sentences that best summarize the document. This is extractive summarization — unlike abstractive summarization, no new text is generated. Instead, existing sentences are scored and ranked by their centrality within the document's semantic structure.

The core insight is that a sentence which is similar to many other sentences in the document is likely to be a good summary sentence — it captures themes that recur throughout the speech.

Technical Methodology

The Five-Stage Pipeline

The summarization system follows a five-stage pipeline, each building on the previous:

#	Stage	Function	What It Does
1	**Tokenize**	`GetSpeech()`	Parse raw text into 90 sentences using NLTK
2	**Vectorize**	`GetDTM()`	Build TF-IDF matrix (90 sentences x 534 terms)
3	**Correlate**	`GetGramian()`	Compute Gramian matrix of sentence similarities
4	**Graph**	`GetGraph()`	Build NetworkX graph from similarity matrix

5	**Rank**	`RankSents()`	Apply PageRank to score and rank sentences

Understanding the Math

TF-IDF Vectorization

Each sentence is converted into a vector in 534-dimensional term space. TF-IDF weighting ensures that distinctive words (like “infrastructure” or “administration”) receive higher weights than common words, creating a meaningful geometric representation where similar sentences point in similar directions.

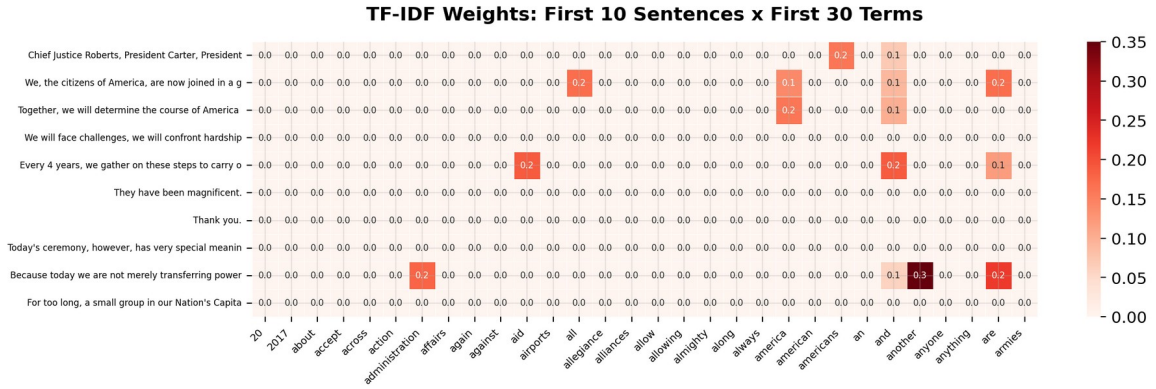

Figure 1: TF-IDF weights for the first 10 sentences and 30 terms. Most cells are zero (sparse matrix).

Gramian Matrix (Cosine Similarity)

The Gramian matrix $G = A \cdot A^T$ computes the dot product between every pair of sentence vectors. Since TF-IDF vectors are non-negative and L2-normalized, this is equivalent to cosine similarity — a value of 1.0 means identical term distributions, 0.0 means no shared vocabulary.

> Key Insight: The Gramian matrix is always symmetric (similarity of A to B equals B to A) and has 1.0 on the diagonal (every sentence is perfectly similar to itself). These properties make it ideal for constructing an undirected similarity graph.

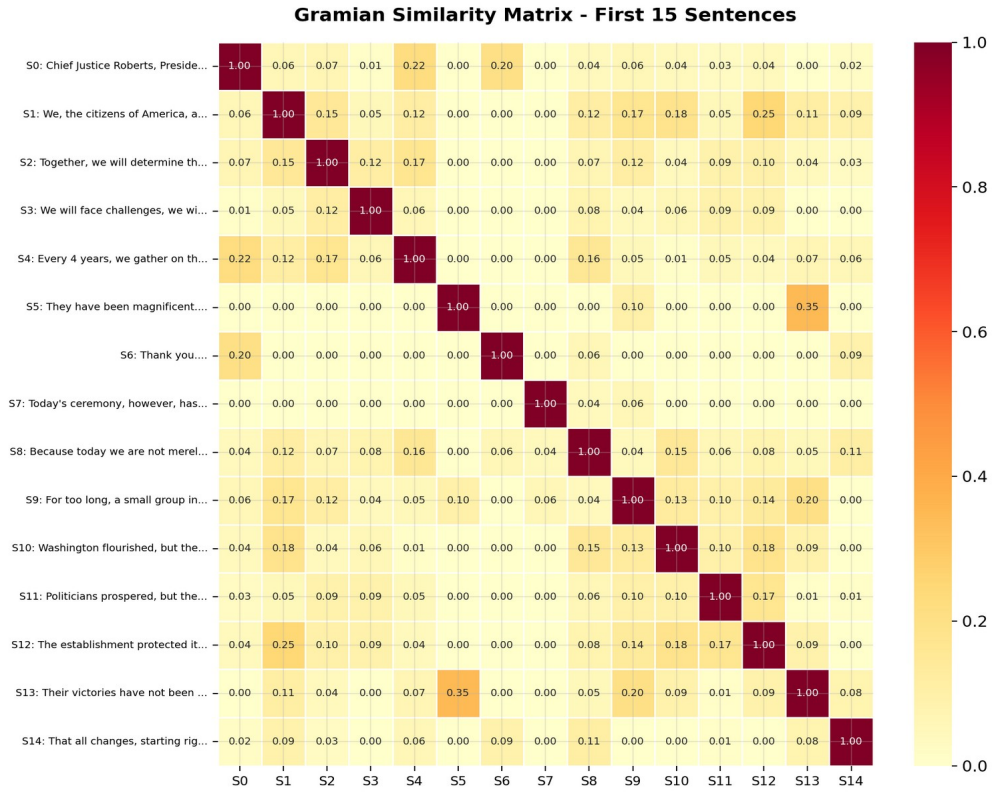

Figure 2: Gramian similarity matrix for the first 15 sentences. Darker cells indicate higher similarity.

PageRank on Sentence Graphs

Google's PageRank algorithm was originally designed to rank web pages by link structure. Here, we apply the same principle to sentences: a sentence is "important" if it is connected to (similar to) many other important sentences. The algorithm iteratively propagates importance scores through the graph until convergence.

The resulting graph has 90 nodes (sentences) and 2,847 edges (non-zero similarities). The average node degree is 63.3, meaning each sentence shares vocabulary with roughly 70% of all other sentences — reflecting the thematic coherence of an inaugural speech.

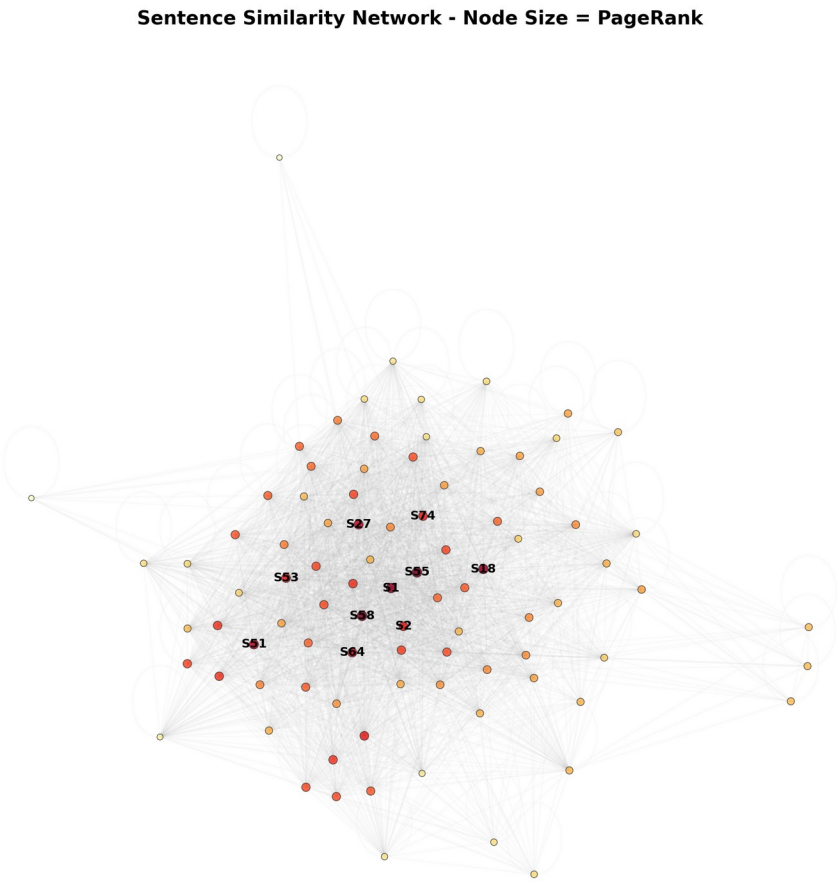

Figure 3: Sentence similarity network. Node size proportional to PageRank score. Top 10 sentences labeled.

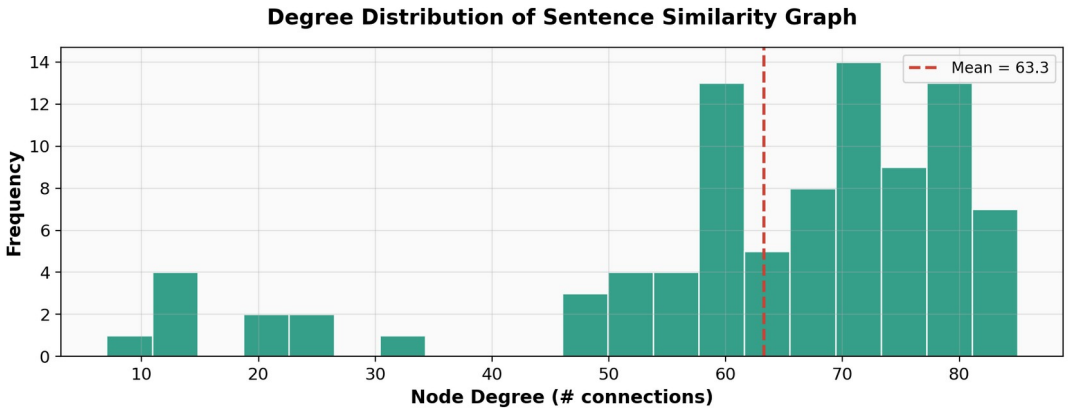

Figure 4: Degree distribution of the similarity graph. Most sentences connect to 60-80 others.

Implementation

Task 1: Sentence Tokenization

```
def GetSpeech(fid='2017-Trump.txt'):
    text = nltk.corpus.inaugural.raw(fid)
    LsSents = nltk.sent_tokenize(text)
    return LsSents
```

Task 2: TF-IDF Document-Term Matrix

```
def GetDTM(LsSents=['']):
    tv = TfidfVectorizer()
    sm = tv.fit_transform(LsSents)
    dfDTM = pd.DataFrame(sm.toarray(),
                         index=[s[:50] for s in LsSents],
                         columns=tv.get_feature_names())
    return dfDTM
```

Task 3: Gramian Similarity Matrix

```
def GetGramian(dfDTM):
    arr = dfDTM.values
    dfSim = pd.DataFrame(arr @ arr.T,
                         index=dfDTM.index,
                         columns=dfDTM.index)
    return dfSim
```

Task 4: Graph Construction

```
def GetGraph(mSim):
    G = nx.from_numpy_array(mSim)
    G.name = 'Sentence Similarities'
    return G
```

Task 5: PageRank Sentence Scoring

```
def RankSents(G, LsSents=['']):
    ranks = nx.pagerank(G)
    dfPgRnk = pd.DataFrame({
        'Rank': [ranks[i] for i in range(len(LsSents))],
        'Sent': LsSents})
    dfPgRnk = dfPgRnk.sort_values('Rank', ascending=False)
    return dfPgRnk
```

Results

PageRank Score Distribution

PageRank scores range from 0.0056 to 0.0172, with the uniform baseline at 1/90 = 0.0111. The most central sentences score 55% above the uniform distribution, indicating they are substantially more connected than average:

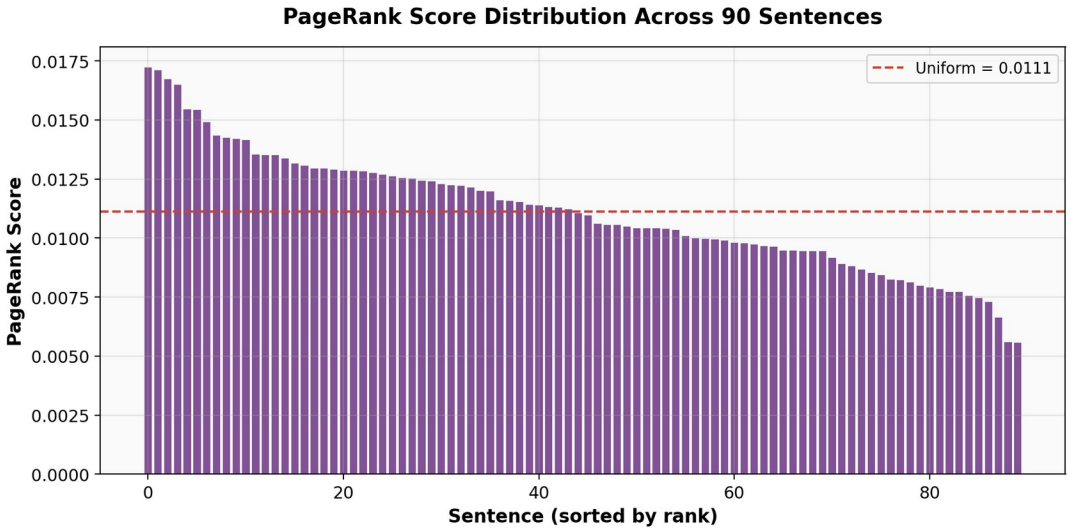

Figure 5: PageRank score distribution across 90 sentences. Red dashed line shows the uniform baseline.

Top Ranked Summary Sentences

The top 10 highest-ranked sentences represent the most thematically central statements in the speech — the sentences that share the most vocabulary with the rest of the document:

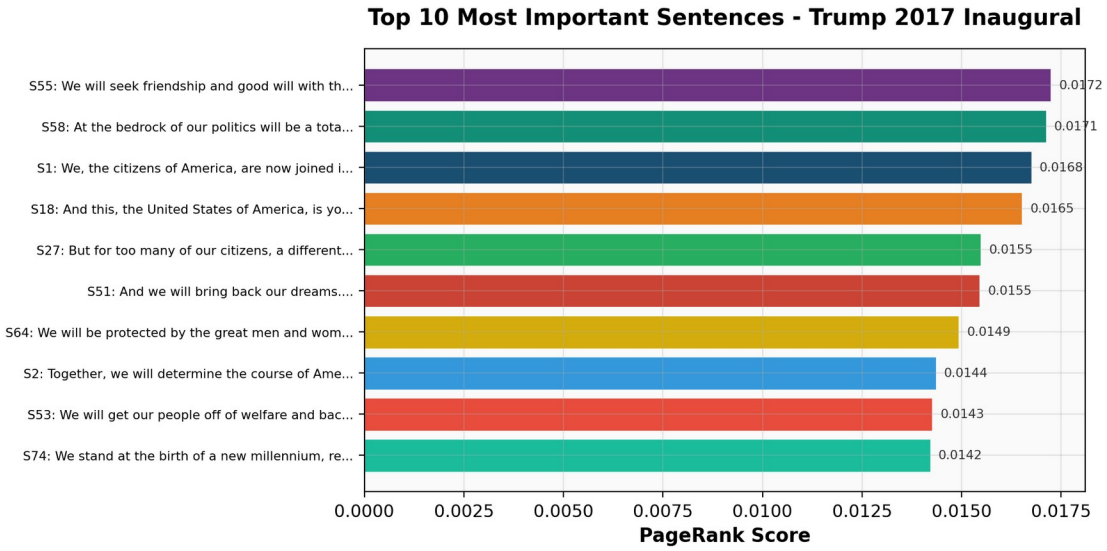

Figure 6: Top 10 sentences by PageRank score from Trump's 2017 inaugural address.

Interpretation: The highest-ranked sentences contain broad thematic language ("people", "country", "America", "dreams") that connects to many other sentences. Narrow, specific sentences (e.g., those mentioning a single policy) rank lower because they share less vocabulary with the rest of the speech.

Technical Skills Demonstrated

Skill Area	Details
NLP & Text Processing	Sentence tokenization, TF-IDF vectorization, sparse-to-dense conversion, text preprocessing
Linear Algebra	Gramian matrix computation ($A \cdot A^T$), cosine similarity, matrix multiplication, symmetric matrices
Graph Theory	Graph construction from adjacency matrices, node degree analysis, network structure
Algorithm Design	PageRank implementation, iterative convergence, centrality-based ranking
Python & Libraries	NLTK, scikit-learn, NetworkX, Pandas, NumPy, Matplotlib, Seaborn
Data Visualization	Network graphs, heatmaps, histograms, bar charts, annotated matrices
Software Engineering	Modular 5-function pipeline, docstrings, type hints, unit-tested code

Extensions & Future Work

- **TextRank Variants:** Implement the original TextRank algorithm (Mihalcea & Tarau, 2004) with edge weights for more nuanced similarity.
- **Sentence Embeddings:** Replace TF-IDF with SBERT embeddings for semantic (meaning-level) rather than lexical (word-level) similarity.
- **Multi-Document Summarization:** Extend the pipeline to summarize across multiple speeches simultaneously.
- **Abstractive Comparison:** Compare extractive PageRank summaries against abstractive summaries from GPT/T5 models.
- **Compression Ratio Control:** Add a parameter to control summary length (e.g., top 5% vs top 20% of sentences).

Conclusion

This project demonstrates how classic graph algorithms can be repurposed for natural language processing. By representing sentences as TF-IDF vectors, computing their pairwise similarities via the Gramian matrix, and applying PageRank to the resulting graph, we built a fully automated extractive summarization system that identifies the most thematically central sentences in any document.

The approach is unsupervised (no training data required), interpretable (every ranking can be traced to specific word overlaps), and modular (each of the five pipeline stages can be independently improved or replaced). These properties make it a practical foundation for real-world summarization applications.